



Playing Guess Who with your kids: Code-word strategy against adversaries ☆

Ami Paz ^{a,*}, Liat Peterfreund ^{b,*}

^a CNRS—LISN & Paris-Saclay University, Gif-sur-Yvette, 91190, France

^b School of Computer Science, Hebrew University of Jerusalem, 9190401, Israel

ARTICLE INFO

Keywords:

Guess Who?

Binary search

Error correcting codes

Erasure codes

Rényi-Ulam games

ABSTRACT

Guess Who is a two-player search game in which each player chooses a character from a deck of 24 cards, and has to infer the other player's character by asking yes-no questions. A simple binary search strategy allows the starting player find the opponent's character by asking 5 questions only, when the opponent is honest.

Real-life observations show that in more realistic scenarios, the game is played against adversaries that do not strictly follow the rules, e.g., kids. Such players might decide to answer all questions at once, answer only part of the questions as they do not know the answers to all, and even lie occasionally. We devise strategies for such scenarios using techniques from error-correcting and erasure codes. This connects to a recent line of work on search problems on graphs and trees with unreliable auxiliary information, and could be of independent interest.

1. Introduction

Guess Who? is a two-player board game where each player aims to guess their opponent's chosen character's identity. Each player has a board that includes cartoon images of 24 characters. At the beginning of the game, each player chooses one of the characters, and the other player's goal is to guess this character. The objective of the game is to be the first player to determine which character was selected by the opponent. Players alternately ask their opponents various yes-no questions on their selected card, such as "Does your character wear glasses?", "Is your character a woman?" etc. The player then eliminates candidates based on the opponent's response by flipping the appropriate characters down. The game ends as soon as there is a player with a single character left on their board.

Nica [1] performed an interesting game-theoretical analysis of the game. By analyzing it as a zero-sum stochastic game, he was able to show that at each round, a player with more cards left should take a "bold move", and ask a question that has the potential of eliminating many characters, while the leading player (with less characters left) should stick to a traditional binary search strategy.

In this work, we take an algorithmic perspective. We hence ignore the game-theoretic competitive considerations that paved the way in Nica's work, and instead, focus only on the underlying algorithmic search problem. To this end, we analyze a version of the game where one player is the *chooser*, who chooses a character and answers questions on it, while the other player is the *guesser*, who has to ask questions in order to correctly guess the chosen character. Thus, the focus of this paper is on strategies for the guesser. Doing so, we follow the footsteps of Knuth [2], who applied a similar approach studying the Mastermind game.

☆ This article belongs to Section A: Algorithms, automata, complexity and games, Edited by Paul Spirakis.

* Corresponding authors.

E-mail address: amipaz.cs@gmail.com (A. Paz).

Naturally, against a well-behaved chooser, the optimal strategy will be a binary search. However, in real-world applications, such as playing Guess Who with young kids, a guesser may need to devise a strategy that yields a correct guess despite a chooser that *deviates* from the game protocol. Inspired by real-life experience, we consider several ways in which a chooser might deviate from the protocol.

A *Not-At-Time-ANswering* (NATAN) adversary is an impatient chooser, which refuses to answer one question at a time, and instead, gives the guesser a single opportunity to present all their questions, and answers them altogether. In terms of theoretical computer science, we can think of this as an asynchronous adversary, or alternatively, we can say that the guesser is *non-adaptive*. In Section 4, we show that there is a simple strategy that allows to perform a binary search even against this adversary, thus guessing in an optimal number of $\lceil \log n \rceil$ questions.

An *Answers Relatively Insecure* (ARI) adversary is a chooser that is having hard time determining if a character has some property, and thus, might answer “I do not know” and not only “Yes” or “No”. This adversary, in addition to modeling a young kid, is also relevant in the context of communication in a noisy channel. Thus, it may not come as a surprise to the reader that in Section 5, we show how to overcome the indecisiveness of the adversary by using *erasure codes*.

Finally, a Valid At Large (VAL) player is a chooser that has a flexible view of the truth. Hence, this chooser might answer incorrectly to a few questions. In a game, we give this player the benefit of the doubt that they are merely mistaken, but in theoretical computer science, we would say we are dealing with a *malicious* player, or with a *Byzantine* agent. Section 6 we study this case: In Section 6.1 we extend the technique used for the ARI adversary by incorporating *error correcting codes*, and propose a strategy for dealing with VAL with an overhead of a few questions. In Section 6.2 we suggest to deal with VAL using an *adaptive* strategy, in which the questions are chosen based on the previous answers. To this end, we notice that playing Guess Who with VAL is a special case of Rényi-Ulam games [3]. Hence, we can apply strategies that were devised for these games, improving upon our non-adaptive strategy from Section 6.1.

The applicability of our approach in real-world scenarios is still to be proven, and an experimental follow-up work is now being considered. Regardless of this aspect, we believe that our work presents simple applications of basic theoretical computer science concepts such as binary search, erasure codes and error correcting codes, and as such can be used for introductory examples for a wide audience.

2. Related work

As mentioned, Nica [1] was the first to conduct a rigorous analysis of the Guess Who game. His work stayed in the framework of the game, and used a stochastic analysis for determining when a player should make a “bold guess”, that in a small probability will result in the elimination of many characters. We note that such an analysis is useless, e.g., against the NATAN adversary, as it crucially relies on the round-based nature of the game.

The NATAN adversary delays all its answers to the end. A more involved scenario, of an adversary which delays its answers by some constant number of questions was considered for the Twenty Questions game [4].

The ARI adversary resembles scenarios in which some of the information is missing. Handling incomplete or missing information is one of the longstanding topics in database research [5–8]. In particular, the study of certain answers (answers that are true regardless of how one completes the missing entries of the queried database) has attracted lots of attention in the database community [9,10].

Searching games in the presence of liars were first studied by Rényi [11] and independently by Ulam [12]. There is a large body of work on the topic since, and we refer the reader to the excellent survey [3] for more details. The VAL adversary presents a setting of a search with partially incorrect information, that is of high research interest in recent years. This is the case, e.g., when studying search and navigation problems in graphs, where the information regarding the target’s location might be unreliable [13–16]. A question even more similar to ours is that of playing twenty questions game with a liar [17], which has tight connections to learning theory.

Guess Who was invented as a simple version of the *Bulls and Cows* game (whose commercial version is known under the name “Mastermind”), another search game where the questions and answers are more involved than the binary questions applied here. In Mastermind, players have to guess their opponent’s 4-peg colorful pattern, while the opponent replying to each guess by specifying the number of correct pegs used in place, and those that are correct but misplaced. A word-based variant of Mastermind is the trendy Wordle game. While the scientific literature on Wordle is rather limited (though numerous analyses, discussions and strategies can be found online), Mastermind was studied in several occasions. In 1977, Knuth [2] devised an optimal deterministic strategy for Mastermind, in only 5 guesses. Interestingly, he analyzed the game as a game with a single guesser and ignored game-theoretic aspects yielding from the fact that the game has two players, as we do here. Chvátal [18] introduced a generalization of Mastermind to k colors and n positions. He studied it from an information-theoretic perspective, and used the probabilistic method to show the existence of a deterministic and asymptotically-optimal strategy for the regime of $k \leq n^{1-\epsilon}$. The case of $k = \Theta(n)$ was later also studied [19] and reduced to coin-weighing problems. Connections between Mastermind (with honest players) and coding theory, specifically uniquely decodable codes for noiseless adder channel, were pointed out in [20]. Several variants [21,22] and approaches to Mastermind were also studied. Mastermind is also used in recent years as a benchmark for evolutionary algorithms [23–27].

3. Formal framework

In this section, we introduce the formal settings based on our algorithmic perspective.

(a) Characters¹

	glasses	no glasses	hat	blond	back hair	brown hair	white hair	glasses and bald
Bill	0	1	0	0	0	0	0	0
Charles	0	1	0	1	0	0	0	0
Claire	1	0	1	0	0	0	0	0
Joe	1	0	0	1	0	0	0	0
Maria	0	1	1	0	0	1	0	0
Max	0	1	0	0	1	0	0	0
Sam	1	0	0	0	0	0	1	1
Susan	0	1	0	0	0	0	1	0
Tom	1	0	0	0	1	0	0	1

(b) A corresponding $\mathbf{T}_{9 \times 8}$ matrixFig. 1. Example of figures and some of their traits. (¹Image taken from [iSLCOLLECTIVE.com](https://www.islcollective.com/).)

3.1. Characters and traits

We denote the number of *characters* by n , and refer to each character by a unique identifier in the set $[n] := \{1, \dots, n\}$. We denote the possible (physical) *traits* of the characters by $[m]$. A trait can include characteristics such as being brunette or not wearing a hat. It may also refer to a combination of physical characteristics, such as being a brunette who does not wear a hat. Since players can only ask yes-no questions, we assume that all possible questions are of the form

$$q(j) := \text{Does your character possess trait } j?$$

where $1 \leq j \leq m$.

The information on the different characters and their traits can be represented as a binary matrix whose rows correspond with characters' identifiers, and columns with traits. Formally, the *traits' matrix*, is a binary $n \times m$ matrix $\mathbf{T}$, where $M_{i,j} = 1$ if character i possesses trait j , and $M_{i,j} = 0$ if character i does not possess trait j . In this way, every row of the matrix describes a character.

Example 1. Fig. 1a presents a subset of characters of the original game and Fig. 1b a corresponding matrix with some of the possible traits listed as its columns. Note that a column may refer to several physical characteristics (e.g., the last column).

3.2. Distinguishability

We assume that every two different characters are *distinguishable*, that is, have at least one different trait. Formally, for each $i \neq i' \in [n]$ there is $j \in [m]$ such that $\mathbf{T}[i][j] \neq \mathbf{T}[i'][j]$. This ensures that no matter which character was chosen by the chooser, the guesser can ask questions on its traits until revealing its identity.

In fact, similarly to [1], we assume even a stronger assumption — not only do every two characters are distinguishable, but also any subset of characters is distinguishable from the rest. We formalize this as follows:

Assumption 1. For any subset $A \subseteq [n]$ of characters, there is a trait j_A such that $A = \{i \in [n] \mid \mathbf{T}[i][j_A] = 1\}$.

In such a case, we say that the trait j_A *separates* A .

Example 2. The trait that separates the subset consisting of Sam and Tom from the rest is “having glasses and being bald” — a Boolean combination of basic physical characteristics. For Sam, Tom, Joe and Claire, it is simply “having glasses”.

We note that Assumption 1 may require composing several physical characteristics, which is sometimes unnatural when playing with kids. We discuss this further in Section 7, where we also present a slightly less demanding assumption, namely that only sets

containing half the characters are separable. A more comprehensive analysis, which relies solely on traits distinguishable by ‘basic’ physical characteristics, must consider the specific version of the game and the question of which characteristics are deemed basic by the players. These issues are too detailed to be addressed in this work; in fact, it is unclear whether anything generic can be said on the game when the restrictions are so specific.

3.3. Games, runs, and classical strategies

We view an n -character game, or, simply, a game, as a process in which a player, namely *the guesser*, needs to search in the n -characters’ space the character chosen by the other player, namely *the chooser*. Classically, this search is done by eliminating elements from $[n]$ until reaching a single one. For simplicity, we assume that a player does not need to explicitly guess the other player’s character after narrowing down the list of possible characters to one; on the other hand, a player cannot guess a character if its list is larger than one. Formally, we define a *run* ρ as a sequence:

$$q_1, a_1, I_1, \dots, q_k, a_k, I_k$$

where for every $1 \leq i \leq k$ it holds that $I_i \subseteq [n]$, q_i and a_i are, respectively, the question and answer of round i of the game, and the following hold for every $1 \leq \ell \leq k$:

- if $q_\ell = q(j)$ and $a_\ell = \text{yes}$ then $I_\ell = \{i \in I_{\ell-1} \mid \mathbf{T}[i][j] = 1\}$;
- if $q_\ell = q(j)$ and $a_\ell = \text{no}$ then $I_\ell = \{i \in I_{\ell-1} \mid \mathbf{T}[i][j] = 0\}$,

where we set $I_0 := [n]$. Notice that the set I_k consists of the characters that are consistent with answers $a_1, \dots, a_k$.

We say that $q_1, \dots, q_k$ defines ρ . In addition, we say that ρ is *successful* if I_k is a singleton. If ρ is successful, we say that $q_1, \dots, q_k$ defines a successful run.

Example 3. Consider the run $q_1, a_1, I_1, q_2, a_2, I_3$ where I_0 consists of all characters that appear in Fig. 1a, $q_1 = q(\text{hat})$, $a_1 = \text{yes}$, $I_1 = \{\text{Claire}, \text{Maria}\}$, $q_2 = q(\text{brown hair})$, $a_2 = \text{yes}$, and $I_3 = \{\text{Maria}\}$.

A *classical strategy* is a procedure that, given a run $q_1, a_1, I_1, \dots, q_i, a_i, I_i$, either chooses the next question q_{i+1} , or terminates if the run is successful. If, for every chosen character, the procedure terminates after choosing at most k questions, we say it is a k -question strategy.

3.4. Our gallery of adversaries

In this paper we consider several adversaries who deviate from the standard game protocol. In order to play with each of them, we have to adjust the notion of a strategy.

Impatient NATAN The first adversary we consider is the *impatient* adversary denoted NATAN, which answers all the questions at once. A run against such an adversary is a normal run, but the strategy is non-adaptive. Formally, a k -question *predetermined-strategy* is a set $\{q_1, \dots, q_k\}$ of questions where $q_\ell := q(j_\ell)$, such that the following set is a *singleton*:

$$\{i \mid \forall \ell : ((a_\ell = \text{yes}) \text{ implies } \mathbf{T}[i][j_\ell] = 1 \text{ and } (a_\ell = \text{no}) \text{ implies } \mathbf{T}[i][j_\ell] = 0)\}$$

where a_ℓ is a correct answer to q_ℓ for every ℓ . That is, there is always a single character determined by answers to these questions. Alternatively, using our previous definitions, we can define k -question *predetermined-strategy* as a set $\{q_1, \dots, q_k\}$ of questions, such that each sequence obtained by a permutation of its elements defines a successful run.

Clueless ARI The second type of adversaries we consider is *clueless* adversaries that might occasionally say they do not know the answer. ARI is such an adversary that might answer “don’t know” at most once. To deal with clueless adversaries, we slightly extend the definition of runs by allowing $a_\ell = \text{don’t know}$, and setting $I_\ell = I_{\ell-1}$ in this case. A *strategy* against a clueless adversary is defined similarly to a classical strategy, while considering the extended notion of run. It is said to be a k -question *strategy* if it terminates after at most k questions for any chosen character.

In fact, our strategy for the clueless adversary is even more elaborate, and holds against an adversary that is *both impatient and clueless*. As the reader may expect, this adversary answers all the questions at once, and might answer “don’t know” on some fraction of them. We define the notion of k -question *predetermined strategy* similarly as before. Note that the alternative definition based on successful runs is valid also here by considering the extended notion of run.

Liar VAL Finally, we consider *liar* adversaries. VAL, for example, is such an adversary that can lie at most once in a game.

A k -question *predetermined strategy* for an impatient liar that lies at most d times is a set $\{q(j_1), \dots, q(j_k)\}$ of questions such that there is a unique $i \in [n]$ for which

$$d_{\text{HAM}}((b_1, \dots, b_k), v^i) \leq d$$

where d_{HAM} stands for the Hamming distance (that is, the number of entries in which two vectors differ), $v^i := (\mathbf{T}[i][j_1], \dots, \mathbf{T}[i][j_k])$, a_ℓ is the answer to $q(j_\ell)$, and $b_\ell := \begin{cases} 1 & a_\ell = \text{yes} \\ 0 & a_\ell = \text{no} \end{cases}$. Intuitively, since the adversary lies, the guesser can only “approximate” the

chosen character, i.e., find a vector that differs from that of the chosen character by at most d bits. The uniqueness of i ensures that the guesser can determine who is the chosen character based on the approximate vector she gets.

4. Playing with honest (but sometimes lazy) players

We start by considering settings in which the chooser is honest, i.e., always answers correctly. The following result is based on the very basic strategy of binary search. In the following, we refer to $\log_2$ simply as $\log$, and, by a slight abuse of notation, we replace “yes” with 1 and “no” with 0.

Theorem 1. *For every n -character game, there is a $\lceil \log n \rceil$ -question strategy. In addition, there is no k -question strategy with $k < \lceil \log n \rceil$.*

Proof. The theorem is achieved by a simple binary search algorithm, defined recursively. Given a set I_ℓ with $|I_\ell| > 1$, we choose $A \subseteq I_\ell$ with $|A| = \lfloor |I_\ell|/2 \rfloor$, and use Assumption 1 to find a trait j with $A = \{i \mid \mathbf{T}[i][j] = 1\}$. We ask $q_{\ell+1} = q(j)$ to get an answer $a_{\ell+1}$, and set $I_{\ell+1} = \{i \mid \mathbf{T}[i][j] = a_{\ell+1}\}$.

It is immediate to see that the algorithm defines a successful run $q_1, a_1, I_1, \dots, q_k, a_k, I_k$ with $|I_k| = 1$. Indeed, using the inequality $|I_{\ell+1}| \leq \lceil |I_\ell|/2 \rceil \leq (|I_\ell| + 1)/2$, a simple induction, and the fact that $|I_0| = n$, we get $|I_k| \leq \frac{n+2^k-1}{2^k}$. For $k = \lceil \log n \rceil \geq \log n$, this implies $|I_k| \leq 2 - 1/n$ and the integrality of $|I_k|$ guarantees $|I_k| = 1$ as claimed.

For the second part of the theorem, let us assume that there is a k -question strategy with questions $q_1, \dots, q_k$. Each successful run that is consistent with this strategy is of the form $I_0, q_1, a_1, I_1, \dots, q_k, a_k, I_k$ where for every ℓ , $a_\ell \in \{\text{yes}, \text{no}\}$ and $I_{\ell+1} \subseteq I_\ell$. Note that $I_{\ell+1}$ is uniquely determined by $a_1, \dots, a_\ell$ and thus the number of possible I_k s is bounded by the number of possible vectors $a_1, \dots, a_k$. Hence, there are at most 2^k successful runs that are consistent with this strategy. If $k < \lceil \log n \rceil$ then $2^k < n$ which implies that $q_1, \dots, q_k$ cannot be a k -question strategy (since there are n characters). $\square$

Notice that this strategy is adaptive, that is, at each round the guesser chooses the next question based on the outcome of the previous one. Is it possible to find a strategy that is not adaptive, that is, a strategy against an impatient adversary? We answer this in the affirmative.

Theorem 2. *For every n -character game, there is a $\lceil \log n \rceil$ -question predetermined-strategy.*

Proof. For $1 \leq \ell \leq \lceil \log n \rceil$, let $A_\ell = \{i \in [n] \mid i[\ell] = 1\}$, where $i[\ell]$ is the bit in location ℓ of the binary expansion of i (whose first bit is $i[1]$). For each ℓ in the range, we set q_ℓ to be the trait separating A_ℓ , which exists by Assumption 1.

To prove this strategy always assures termination in $\lceil \log n \rceil$ questions, we consider the run $q_1, a_1, I_1, \dots, q_{\lceil \log n \rceil}, a_{\lceil \log n \rceil}, I_{\lceil \log n \rceil}$ induced by the aforementioned questions. The choice of questions guarantees $I_\ell = \{i \mid i[1] = a_1, \dots, i[\ell] = a_\ell\}$. Therefore, $I_{\lceil \log n \rceil}$ contains solely the element i with binary encoding $a_{\lceil \log n \rceil} a_{\lceil \log n \rceil - 1} \dots a_2 a_1$. $\square$

This “static” approach can be useful when playing with a *Not-At-Time-ANswering* (NATAN) player that answers the questions altogether. Therefore, we can conclude the following.

Corollary 1. *The classical Guess Who game has a 5-question predetermined-strategy, i.e., a strategy that works even against NATAN.*

5. Playing with clueless players

In this section, we consider *clueless* players, that do not always know the answer to a question. A clueless player might answer, in addition to “yes” and “no”, the answer “I do not know” (“don’t know”, for brevity). One way to circumvent this answer is, whenever the player answers “don’t know” on some trait, to ask a question on a different trait that has exactly the same answers on all characters. However, this requires the assumption that such a trait always exists, and in addition, it does not work against an adversary that is both clueless and impatient. Instead, we suggest a different solution, using *erasure codes*.

Definition 1 ([28]). An *erasure code* that can handle e erasures is a collection C of binary vectors, namely *code-words*, such that there is a recovery algorithm $\mathcal{A}$ that gets as input $c \in C$ with at most e locations replaced by “?”, and returns c .

We restrict our attention to codes where all the code-words have equal length. A simple erasure code with $e = 1$ can be achieved, for example, by adding a parity bit—the recovery algorithm in this case will replace the “?” with a value complementing the number of 1-s in the vector to be even.

Before presenting the result, let us describe its formal setup.

Theorem 3. *If there is an erasure code composed of at least n binary vectors of length k , that can handle e erasures, then for every n -character game there is a k -question predetermined-strategy against an adversary that answers “don’t know” at most e times.*

Proof. Assign each character i with a unique vector v_i from the code. For $1 \leq \ell \leq k$, let $A_\ell = \{i \mid v_i[\ell] = 1\}$ be all the characters with the ℓ -th entry of their vector equal to 1. Let j_ℓ be the trait that separates A_ℓ from $\bar{A}_\ell$, which exists by Assumption 1.

We use the question sequence $q_1 := q(j_1), \dots, q_k := q(j_k)$, which defines a run $q_1, a_1, I_1, \dots, q_k, a_k, I_k$. From the sequence $(a_1, \dots, a_k)$ of answers, which contains at most e answers of “?”, we find the unique way to complement the vector into a code-word $v' = (a'_1, \dots, a'_k)$. The character i with $v_i = v'$ is then the desired character.

For correctness, consider the true character i^* chosen, and the sequence $(a_1^*, \dots, a_k^*)$ of the true answers to the questions, where a_ℓ^* is the answer on $q(j_\ell)$. Note that the choice of questions guarantees that the following are equivalent: $a_\ell^* = 1$; $i^* \in A_\ell$; and $v_{i^*}[\ell] = 1$. This concludes in $v_{i^*}[\ell] = a_\ell^*$, from which we have $v_{i^*} = (a_1^*, \dots, a_k^*)$. By the assumption of at most e answers are missing in the answer vector $(a_1, \dots, a_k)$, and the properties of the erasure code ensure that the completion of this vector is unique, and is v_{i^*} . Hence, the algorithm returns i^* , as desired. $\square$

One application of the above theorem is when the adversary may answer “don’t know” at most once. For this, we use simple parity check bit. That is, by concatenating a parity bit to every $\lceil \log n \rceil$ -bit binary vector, we get the following code.

Lemma 1. *For any positive integer n , there is an erasure code of n vectors of length $\lceil \log n \rceil + 1$ allowing to overcome a single deletion.*

Recall that ARI is a player that might answer “don’t know” at most once. By choosing $n = 24$ in the lemma and applying Theorem 3, we get the following corollary.

Corollary 2. *The classical game of Guess Who has a 6-question predetermined-strategy against ARI.*

What about ARI-like adversaries who can answer “don’t know” at most twice or three times? For that we use the Gilbert-Varshamov bound [28, §4.3]:

Lemma 2 (Gilbert-Varshamov bound). *For positive integers k, N satisfying $\sum_{j=0}^{d-2} \binom{k-1}{j} < 2^{k-N}$, there exists a distance- d linear code composed of at least 2^N binary vectors of length k .*

A distance- d code enables to overcome at most $d - 1$ erasers [28, §1.5.3], which together with Theorem 3 enables us to conclude the following extension of Corollary 2:

Corollary 3. *The classical game of Guess Who has a 9-question and 11-question predetermined-strategies against an ARI-like adversary who says “don’t know” at most twice and 3 times, respectively.*

6. Playing with liars

While in the previous section, the clueless player was honest (i.e., provided only correct answers), we now turn our attention to a player that might give incorrect answers. These are sometimes called Byzantine or malicious players, but we will give the kids the benefit of the doubt and say they are merely “mistaken”.

6.1. Predetermined strategies with VAL

We start by considering a player with incorrect answer that is also “lazy”, i.e., answers all the questions at once. A key tool for this will be *error correcting codes*.

Definition 2. An *error correcting code* of distance d is a collection of equal-length binary vectors, such that the Hamming distance between every two is at least d .

We will refer to such a code a “distance- d code”. Note that, given such a code and a vector v , if there exists a code word c with Hamming distance at most $\lfloor \frac{d-1}{2} \rfloor$ from v , then c is the unique code-word with this property. The common use of these codes is thus by “correcting” each such vector v to the unique nearest code-word c ; hence, we say that a distance- d code can *fix* at most $\lfloor \frac{d-1}{2} \rfloor$ errors.

Theorem 4. *If there is a distance- d error correcting code composed of at least n binary vectors of length k , then for every n -character game there is a k -question predetermined-strategy against an adversary that performs at most $\lfloor \frac{d-1}{2} \rfloor$ mistakes.*

Proof. Assign each character i with a unique vector v_i from the code. For $1 \leq \ell \leq k$, let $A_\ell = \{i \mid v_i[\ell] = 1\}$ be all the characters with the ℓ -th entry of their vector equal to 1. Let j_ℓ be the trait that separates A_ℓ , which exists by Assumption 1.

We use the question sequence $q_1 = q(j_1), \dots, q_k = q(j_k)$, which defines a run $I_0, q_1, a_1, I_1, \dots, q_k, a_k, I_k$. From the sequence $(a_1, \dots, a_k)$ of answers, we find the unique closest code-word $v' = (a'_1, \dots, a'_k)$, which differs from the sequence of answers by at most $\lfloor \frac{d-1}{2} \rfloor$ answers. The character i with $v_i = v'$ is then the desired character.

For correctness, consider the true character i^* chosen, and the sequence $(a_1^*, \dots, a_k^*)$ of the true answers to the questions, where a_ℓ^* is the answer on $q(j_\ell)$. Note that the choice of questions guarantees that the following are equivalent: $a_\ell^* = 1$; $i^* \in A_\ell$; and $v_{i^*}[\ell] = 1$. This concludes in $v_{i^*}[\ell] = a_\ell^*$, from which we have $v_{i^*} = (a_1^*, \dots, a_k^*)$. By the assumption of at most $\lfloor \frac{d-1}{2} \rfloor$ wrong answers, we know that the answer vector $(a_1, \dots, a_k)$ differs from v_{i^*} in at most $\lfloor \frac{d-1}{2} \rfloor$ positions, which implies that the unique vector closest to the answer vector is indeed v_{i^*} , and the algorithm returns i^* as desired. $\square$

One application of the above theorem is for VAL adversary. For this, we use a code guaranteed to exist by the Gilbert-Varshamov bound as appears in Lemma 2. By choosing $k = 9$, $N = 5$ and $\lfloor \frac{d-1}{2} \rfloor = 1$ above, we get a code that implies, using Theorem 4, the following corollary.

Corollary 4. *The classical game of Guess Who has a 9-question predetermined strategy against VAL.*

Notice that here, unlike the ARI-like adversary case in which the distance between every two code-words is exactly the number of “don’t knows”, for VAL-like adversaries we need distance $2m + 1$ where m is the maximum number of lies. That is, the number of additional questions in a winning strategy against ARI is equivalent to the number of “don’t know” but against VAL it is two times plus one the number of lies. The intuition is that while playing with VAL we do not know which of the answers are lies. We can generalize Corollary 4 to VAL-like adversaries who lie at most m times. In particular, by setting $N = 5, k = 14, m = 2$ and $N = 5, k = 19, m = 3$, respectively, we get the following.

Corollary 5. *The classical game of Guess Who has a 14 and 19-question predetermined strategies against VAL-like adversaries who lie at most twice and three times, respectively.*

6.2. Adaptive strategies with VAL

In this section we show that the number of questions when playing with a liar can be reduced if the liar is not “lazy”, i.e., if the questions can be adapted in an online manner.

An adaptive error correcting code is a code designed, e.g., for a communication system with a reliable and immediate feedback channel. That is, the sender learns the value that arrived at the receiver before choosing its next bit. This can model, e.g., a sender which sends weak signals (a satellite) communicating with a strong receiver (earth).

Adaptive error correcting codes are known to be relevant to searching games with errors, and specifically to *Rényi-Ulam games*. The relevant variant of these games is defined next.

Definition 3. A *Rényi-Ulam game* is a game between a guesser and a chooser. The chooser has an element from a set $[n]$, and the guesser repeatedly asks questions of the form “is the element in A ?”, for $A \subseteq [n]$. The chooser answers the questions, and may lie on at most a predetermined number m of them. The goal of the guesser is to find the element in as few questions as possible.

Guess Who with a liar is a Rényi-Ulam game, hence results for the latter apply to our setting as well. We now recall some results about Rényi-Ulam games, starting with a lower bound by Berlekamp [29]. He defined the *volume* of a state of the game, from which he derives the following bound.

Lemma 3 ([29]). *If a k -question strategy for the Rényi-Ulam game with m lies exists then*

$$n \sum_{i=0}^m \binom{k}{i} \leq 2^k.$$

Interestingly, this value is achievable for some values of n and m but not for others. The exact number of questions needed for Rényi-Ulam games of arbitrary sizes are only known for one, two, and three lies. Tight results are also known for specific and large sizes n of the search space, but these are of less interest to us. See the excellent survey [3] for details.

With a single lie, the volume bound is achievable for even values of n , but not for odd ones.

Lemma 4 ([30, p. 136]). *A k -question strategy for the Rényi-Ulam game with a single lie exists: for even n , if and only if $n(k+1) \leq 2^k$ for odd n , if and only if $n(k+1) + k - 1 \leq 2^k$.*

With two lies the volume bound is achievable when n is divisible by four, while other values of n require more questions.

Lemma 5 ([31, Theorem 6.1]). *If $n \geq 14$ is divisible by 4 then a k -question strategy for the Rényi-Ulam game with two lies exists if and only if $n(k^2 + k + 2) \leq 2^{k+1}$.*

With three lies, two questions more than the volume bound are necessary for $n \in \{3, 5\}$, and otherwise only one extra question is needed, or no extra question at all. The exact characterization is rather involved, but for the case $n = 24$ the bound is exactly achievable.

Lemma 6 ([32, Theorem 40]). *For $n = 24$, a k -question strategy for the Rényi-Ulam game with three lies exists if and only if $n \frac{k^3 + 5k + 6}{6} \leq 2^k$. Specifically, there is a 14-question strategy in this case.*

These give results for Guess Who which improve upon the predetermined strategies, as follows.

Corollary 6. *The classical game of Guess Who has an 8, 11 and 14-question adaptive strategies against VAL-like adversaries who lie at most once, twice or three times, respectively.*

7. Asking simple questions

Recall that Assumption 1 asserts that any set of characters can be separated from the rest of the characters by a single question. While this is generally true (unless two characters are identical), a real-world application of this assumption might results in cumbersome questions, such as “does your character have glasses and is bald?” Simple questions composed of basic physical characteristics are clearly preferable, but it is not always clear which physical characteristics are basic — this may depend, e.g., on the language spoken and the level of sophistication of the players. Not only that, but note also that different versions of the game (e.g. different publisher or year) may allow different simple questions. Hence, different assumptions in the spirit of Assumption 1 may hold using simple questions in some settings but not in others.

Nevertheless, any version must allow some sets of characters to be separated from the others. A natural weakening of Assumption 1 asserts that only sets composed of *half the characters* can be separated from the rest by a single simple question. This is generally the trend in the common game boards (by Hasbro): for example, half the characters in the new boards are women, compared to 5 out of 24 in older versions. Indeed, this assumption does not guarantee that *any* balanced set can be separated by simple questions, and has no asymptotic effect on the required number of separable sets. Yet, such “balanced” questions are more natural, and considering them also sheds light on a nice property of error-correcting codes. As mentioned above, it seems impossible to say something generic about Guess Who if we restrict the questions much further. We formalize the new assumption as follows.

Assumption 2. For any subset $A \subseteq [n]$ of $|A| = \lfloor \frac{n}{2} \rfloor$ characters, there is an atomic trait j_A such that $A = \{i \in [n] \mid \mathbf{T}[i][j_A] = 1\}$.

An *atomic trait* is a trait which can be asked upon by a *simple question*. As discussed, both terms depend on the context, and so is this assumption.

Many error correcting codes, including the ones whose existence is promised by the Gilbert-Varshamov bound, are *linear codes*. A code is called linear if any linear combination of its code-words is also a code-word. We now prove another property of linear codes, which we then use in order to replace Assumption 1 by Assumption 2 in all our predetermined strategies.

Lemma 7. *Consider a binary linear code C composed of binary words of length k , and an integer $1 \leq \ell \leq k$. Then the set $A_\ell = \{v_i \in C \mid v_i[\ell] = 1\}$ satisfies $|A_\ell| = 0$ or $|A_\ell| = |C|/2$.*

Proof. Consider a linear code as above, with $|C| = 2^N$ code-words. By rearranging the entries of the code-words we can assume that any possible combination of the first N entries exists in some code-word (and only in one such word), while any other entry is a linear combination of the first N entries. Put differently, the set of code-words is the image of a linear transformation $\mathbb{F}_2^N \rightarrow \mathbb{F}_2^k$ represented by a matrix of the form

$$(I \mid M)$$

where I is the $N \times N$ identity matrix and M is a binary $N \times (k - N)$ matrix. For further discussion of the above properties see, e.g., [28, §2.2].

Hence, any code-word $c \in C$ has the following structure: any coordinate $c[\ell]$ with $1 \leq \ell \leq N$ can get any value, independently of the values of any other coordinate $1 \leq \ell' \leq N$. On the other hand, any coordinate $N < \ell \leq k$ is the sum of a subset of the first N coordinates, i.e., for any such ℓ either $c[\ell] = 0$ or $c[\ell] = c[i_1] + \dots + c[i_t]$ with $1 \leq i_1 \leq \dots \leq i_t \leq N$.

Hence, for any coordinate $1 \leq \ell \leq N$, we have $c[\ell] = 1$ in exactly $|C|/2$ of the cases, and the claim holds for such values of ℓ . For $N < \ell \leq k$, either $c[\ell] = 0$ and $|A_\ell| = 0$ as claimed, or $c[\ell] = c[i_1] + \dots + c[i_t]$. We now prove that in the latter case, $c[\ell] = 1$ in exactly $|C|/2$ of the code-words.

Fix $N < \ell \leq k$ with $c[\ell] = c[i_1] + \dots + c[i_t]$, $t \geq 1$. Define a mapping $f : C \rightarrow C$ by mapping each code-word $c \in C$ to the word with the same first N coordinates, except for $c[i_1]$ which is flipped. That is,

$$f : c[1], \dots, c[i_1], \dots, c[N], c[N+1], \dots, c[k] \mapsto \\ c[1], \dots, 1 - c[i_1], \dots, c[N], c'[N+1], \dots, c'[k]$$

where $c'[N+1], \dots, c'[k]$ are some linear combinations of $c[1], \dots, c[N]$.

As $c[\ell] = c[i_1] + \dots + c[i_\ell]$, we have $c'[\ell] = 1 - c[i_1] + \dots + c[i_\ell] = 1 - c[\ell]$ (over $\mathbb{F}_2$), so f flips the ℓ -th coordinate. Hence, f maps elements of A_ℓ to $C \setminus A_\ell$, and vice versa. Moreover, $f(f(c)) = c$, so f must be injective. The restrictions of f to A_ℓ and to its complement are injective maps from A_ℓ to $C \setminus A_\ell$ and vice versa, hence $|A_\ell| = |C \setminus A_\ell|$ and the proof is complete. $\square$

For executing a binary search, Assumption 2 suffices. The other predetermined strategies we devise in this work consist of questions separating a set A_ℓ of characters, where $A_\ell = \{i \mid v_i[\ell] = 1\}$ is chosen by code-words from some code C . Lemma 7 guarantees that each such A_ℓ has $|A_\ell| = |C|/2$ (or $|A_\ell| = 0$, in which case no question is required), and Assumption 2 assures that such a set can be separated using a simple question. This yields the following extension of our results.

Theorem 5. *If Assumption 2 holds, then all our predetermined strategies hold even when restricted to asking simple questions.*

When the number of characters is not a power of 2, we use a code C with slightly more words than the number of characters, which may even allow to weaken Assumption 2 further. For example, in a game board with 24 characters we use a code of 32 words, so having $|A_\ell| \in \{8, \dots, 16\}$, for different values of ℓ , might suffice. The exact size required for each A_ℓ in this case depends on the code chosen, the 24 code words from C chosen to be used, and the specific adversary.

Note also that the claim must hold only for sets A_ℓ on which questions are asked, so if there is a strategy using 6 questions (which is the case for ARI), only 6 specific values of ℓ must satisfy Assumption 2. Again, the specific values of ℓ depend on the choice of code and code-words from it.

Finally, our adaptive strategies for coping with lies use more subtle techniques. These do not use linear algebra but are built ad-hoc for the number of lies and the size of the search space, so incorporating also limitations on the types of questions seems unrealistic. This extension is left open for the curious reader.

8. Concluding remarks

Sophisticated adversaries One natural question that arises from our work is playing Guess Who against an adversary that disobeys the rules in more than one way. If the adversary is both impatient and clueless then our strategy for playing against a clueless adversary still applies. Similarly, when the adversary is both impatient and mistaken, our predetermined strategy for a mistaken adversary works. Naturally, the adaptive strategy for a mistaken adversary will not work for an impatient-mistaken one.

Can an adaptive strategy for a clueless adversary give better results than our predetermined one? A simple approach for this is to repeat unanswered questions. While this is optimal in theory, e.g., in communication channels, it is an unsatisfactory solution in the Guess Who case. This can be easily circumvented: if the adversary answers “don’t know” in the first round, we can permute the characters and start over. If it answers “don’t know” later, on a set A , we can ask about a set $A \cup \{x\}$ for a character $x \notin A$ which we already know was not chosen.

A more intriguing question arises when the adversary might be both clueless and mistaken (and perhaps also impatient, but as before, this will not result in any further complications). Luckily, this question can also be handled using the tools developed in this paper. A simple strategy against an adversary that might answer “don’t know” at most d_1 times and perform at most d_2 mistakes is by replacing all the “don’t know” answers with arbitrary values, and treating the resulting vector as if it might contain at most $d_1 + d_2$ mistakes (i.e. using a distance- $(2(d_1 + d_2) + 1)$ error correcting code). A more economical strategy against such an adversary is by using the fact that most codes can handle both errors and erasures [33, §3.4] (which will require a distance- $(d_1 + 2d_2 + 1)$ code). The details are very similar to the proofs of Theorems 3 and 4, and are omitted for brevity.

Related games As mentioned, Guess Who was invented as a simple version of the color-guessing Mastermind game. In Mastermind, the questions and answers are more involved than the binary questions asked in Guess Who, and thus, playing it with players that do not strictly follow the game protocol is an interesting direction we leave for future research.

Similar question holds for the trendy Wordle game, where the goal is not to guess a character but a word. Among the gallery of variations of Wordle that can be found online, one can consider a variant that does not always correctly mark the guessed letters, or where the user does not get feedback for the guesses until the end. These will probably not be the most fun-to-play version of the game, but they raise interesting theoretical questions, such as error correction in a natural language.

The future of guess who The commercial version of Guess Who was a target for criticism for its gallery of characters, where most characters are white males, and the exceptions (usually 5 out of 24) are women or people of color. One can consider a more inclusive version of the game, not only by splitting the traits more equally, but also by allowing non-binary answers to some questions. This could be done, e.g., by allowing questions such as “what is the color of the character’s hair”. A trinary version of the game (three hair colors, no-glasses/monocle/glasses, etc.) is yet to be presented, and the corresponding algorithmic questions are yet to be studied.

CRedit authorship contribution statement

Ami Paz: Conceptualization, Formal analysis, Writing – original draft, Writing – review & editing. **Liat Peterfreund:** Conceptualization, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors are grateful to Yonatan, Yuval and Ari for inspiring this work, and to Ben Lee Volk for discussions on error correcting codes.

References

- [1] M. Nica, Optimal strategy in “guess who?”: beyond binary search, *Probab. Eng. Inf. Sci.* 30 (4) (2016) 576–592, <https://doi.org/10.1017/S026996481600022X>.
- [2] D.E. Knuth, The computer as master mind, *J. Recreat. Math.* 9 (1977) 1–6.
- [3] A. Pelc, Searching games with errors - fifty years of coping with liars, *Theor. Comput. Sci.* 270 (1–2) (2002) 71–109.
- [4] A. Ambainis, S.A. Bloch, D.L. Schweizer, Delayed binary search, or playing twenty questions with a procrastinator, *Algorithmica* 32 (4) (2002) 641–651.
- [5] W. Lipski Jr, On databases with incomplete information, *J. ACM* 28 (1) (1981) 41–70.
- [6] T. Imieliński, W. Lipski Jr, Incomplete information in relational databases, in: *Readings in Artificial Intelligence and Databases*, Elsevier, 1989, pp. 342–360.
- [7] R. Rosati, On the decidability and finite controllability of query processing in databases with incomplete information, in: *Proceedings of the Twenty-Fifth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2006, pp. 356–365.
- [8] M. Console, P. Guagliardo, L. Libkin, On querying incomplete information in databases under bag semantics, in: *IJCAI*, vol. 17, 2017, pp. 993–999.
- [9] J. Wijsen, On the first-order expressibility of computing certain answers to conjunctive queries over uncertain databases, in: *Proceedings of the Twenty-Ninth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2010, pp. 179–190.
- [10] C. David, L. Libkin, F. Murlak, Certain answers for xml queries, in: *Proceedings of the Twenty-Ninth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2010, pp. 191–202.
- [11] A. Rényi, On a problem of information theory, *MTA Mat. Kut. Int. Kozl.* 6B (1961) 505–516.
- [12] S. Ulam, *Adventures of a Mathematician*, Scribner, 1976.
- [13] L. Boczkowski, U. Feige, A. Korman, Y. Rodeh, Navigating in trees with permanently noisy advice, *ACM Trans. Algorithms* 17 (2) (2021) 15.
- [14] N. Hanusse, D. Ilcinkas, A. Kosowski, N. Nisse, Locating a target with an agent guided by unreliable local advice: how to beat the random walk when you have a clock?, in: *Proceedings of the 29th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing*, 2010.
- [15] N. Hanusse, E. Kranakis, D. Krizanc, Searching with mobile agents in networks with liars, in: *Euro-Par*, 2000.
- [16] D. Dereniowski, D. Graf, S. Tiegel, P. Uznański, A framework for searching in graphs in the presence of errors, in: *SOSA*, 2019.
- [17] Y. Dagan, Y. Filmus, D. Kane, S. Moran, The entropy of lies: playing twenty questions with a liar, in: *ITCS*, in: *LIPICs*, vol. 185, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 1.
- [18] V. Chvátal, Mastermind, *Combinatorica* 3 (3) (1983) 325–329.
- [19] B. Doerr, C. Doerr, R. Spöhel, H. Thomas, Playing mastermind with many colors, *J. ACM* 63 (5) (2016) 42:1–42:23.
- [20] G. Kabatianski, V.S. Lebedev, J. Thorpe, The mastermind game and the rigidity of the Hamming space, in: *2000 IEEE International Symposium on Information Theory (Cat. No.00CH37060)*, 2000, p. 375.
- [21] G. Ville, An optimal mastermind (4, 7) strategy and more results in the expected case, *ArXiv*, arXiv:1305.1010 [abs], 2013.
- [22] A. Berger, C. Chute, M. Stone, Query complexity of mastermind variants, *Discrete Math.* 341 (3) (2018) 665–671.
- [23] J. Gagneur, M.C. Elze, A. Tresch, Selective phenotyping, entropy reduction, and the mastermind game, *BMC Bioinform.* 12 (2011) 406.
- [24] C. Cotta, J.J.M. Guervós, A.M. García, T.P. Runarsson, Entropy-driven evolutionary approaches to the mastermind problem, in: *PPSN*, 2010.
- [25] J.J.M. Guervós, A.M. García, P.A. Castillo, C. Cotta, M.G. Valdez, A search for scalable evolutionary solutions to the game of mastermind, in: *2013 IEEE Congress on Evolutionary Computation*, 2013, pp. 2298–2305.
- [26] J.J.M. Guervós, P.A. Castillo, A.M. García, A.I. Esparcia-Alcázar, Improving Evolutionary Solutions to the Game of Mastermind Using an Entropy-Based Scoring Method, *GECCO '13*, 2013.
- [27] L. Berghman, D.R. Goossens, R. Leus, Efficient solutions for mastermind using genetic algorithms, *Comput. Oper. Res.* 36 (2009) 1880–1885.
- [28] R. Roth, *Introduction to Coding Theory*, Cambridge University Press, 2006.
- [29] E.R. Berlekamp, Block coding for the binary symmetric channel with noiseless, delayless feedback, in: H. Mann (Ed.), *Error-Correcting Codes*, Wiley, New York, 1968, pp. 61–85.
- [30] A. Pelc, Solution of Ulam’s problem on searching with a Lie, *J. Comb. Theory, Ser. A* 44 (1) (1987) 129–140.
- [31] W. Guzicki, Ulam’s searching game with two lies, *J. Comb. Theory, Ser. A* 54 (1) (1990) 1–19.
- [32] C. Deppe, Solution of Ulam’s searching game with three lies or an optimal adaptive strategy for binary three-error-correcting codes, *Discrete Math.* 224 (1–3) (2000) 79–98.
- [33] S. Lin, D.J. Costello, *Error Control Coding*, vol. 2, Prentice Hall, New York, 2001.